

It's not a FAD: first results in using Flows for unsupervised Anomaly Detection at 40 MHz at the Large Hadron Collider

Francesco Vaselli^{1,2*}, Chang Sun⁵, Thea Aarrestad⁴, Dimitrios Danopoulos¹,
Roope Oskari Niemi¹, Maciej Mikolaj Glowacki¹, Katya Govorkova³, Vladimir Loncar⁶,
Felice Pantaleo¹ and Maurizio Pierini¹

¹European Organization for Nuclear Research (CERN) CH-1211 Geneva 23, Switzerland

²also at Scuola Normale Superiore, Pisa, Italy and Istituto Nazionale di Fisica Nucleare (INFN) sezione di Pisa, Italy

³Massachusetts Institute of Technology, Cambridge, MA, USA

⁴ETH, Federal Institute of Technology Zurich, Zurich, CH

⁵California Institute of Technology, Pasadena, CA, USA

⁶Institute of Physics, Belgrade, Serbia

Abstract

We present the first implementation of a Continuous Normalizing Flow (CNF) model for unsupervised anomaly detection within the realistic, high-rate environment of the Large Hadron Collider's L1 trigger systems. While CNFs typically define an anomaly score via a probabilistic likelihood, calculating this score requires solving an Ordinary Differential Equation, a procedure too complex for FPGA deployment. To overcome this, we propose a novel, hardware-friendly anomaly score defined as the squared norm of the model's vector field output. This score is based on the intuition that anomalous events require a larger transformation by the flow. Our model, trained via Flow Matching on Standard Model data, is synthesized for an FPGA using the hls4ml and da4ml libraries. We demonstrate that our approach effectively identifies a variety of beyond-the-Standard-Model signatures with performance comparable to existing machine learning-based triggers. The algorithm achieves a latency of a few hundred nanoseconds, or even less when using advanced quantization techniques, and requires minimal FPGA resources, establishing CNFs as a viable new tool for real-time, data-driven discovery at 40 MHz.

1 Introduction

The 40 million proton-proton collisions per second at the CERN Large Hadron Collider (LHC) [1] present a formidable data challenge. The sensors in the ATLAS [2] and CMS [3] detectors output hundreds of terabytes of data per second, which must be filtered in real-time to decide what to save on disk for further analysis. This is accomplished by a two-stage trigger system: a hardware-based Level-1 Trigger (L1T) [4, 5, 6, 7] on *Field Programmable Gate Arrays* (FPGAs), which reduces the data rate by almost a factor of 1000 within microseconds, followed by a software-based High-Level Trigger (HLT) on a CPU farm.

The selection algorithms in these triggers are typically supervised, targeting specific physics signatures motivated by theory, as exemplified by the successful search for the Higgs boson [8, 9]. This paradigm, however, may fail to identify unforeseen new physics. Consequently, there is significant community effort to use unsupervised machine learning for model-agnostic searches [10, 11]. These efforts have explored autoencoders [12] for offline process-

ing and have proposed their integration into the HLT to capture rare events in a special data stream [13, 14], echoing earlier non-ML strategies [15].

More recently, a new approach to anomaly detection (AD) argued for deploying these algorithms earlier in the data processing chain: at the L1T. Placing an AD search at this stage avoids the selection biases introduced by the standard L1T algorithms, which often discard potentially interesting low-energy topologies. An unbiased, model-independent AD trigger can select events based on their abnormality, not a specific signature, thus maximizing discovery potential. Govorkova et al. [16] pioneered this new type of strategy using autoencoders as the AD algorithm to be ported to the L1T.

Since 2023, the CMS Collaboration is operating autoencoder-based anomaly detection algorithms. At first, a VAE based on high-level features (named AXOL1TL [17]) was deployed, following the strategy highlighted in Ref [16]. Then, a Convolutional Autoencoder (named CICADA [17]) was put in use, using Knowledge Distillation as a compression algorithm [18]. In 2025, ATLAS took the first steps towards the same strategy, with the deployment of

*Corresponding author: francesco.vaselli@cern.ch

GELATO [19]. All these algorithms consider autoencoders as the baseline tool for anomaly detection in real-time, as originally proposed in Ref. [16].

To the best of our knowledge, the present work is the first proof of concept for the application of a new type of ML model at the L1T for unsupervised AD: *Continuous Normalizing Flows* (NF) [20].

Discrete NF have already been considered as anomaly detection tools for high energy physics, but only in the context of offline data analysis, see [11, 21, 22].

Implementing these algorithms at the L1T, with its severe latency ($< 1 \mu\text{s}$) and resource constraint, is made possible by open-source `hls4ml` [23, 24, 25, 26] and `da4ml` [27] libraries, providing a pathway to deploy neural networks and other ML models on FPGAs. By generating highly optimized, fully on-chip firmware, these tools can meet the L1T latency and throughput requirements (initiation interval $< 150 \text{ ns}$, related to the bunch-crossing time for the LHC operations). Moreover, they support quantization-aware training (QAT) [28] that potentially allows extreme model compression and a further reduction of FPGA resource consumption.

The main contributions of this work are the following:

- We train a Continuous NF on a realistic dataset of low-level features of physics objects Standard Model (SM) signatures, and from there we define an *anomaly detection score* suitable for inference on FPGA;
- We evaluate results on different benchmarks for new physics scenarios at Floating Point precision, comparing it to a similar architecture of Govorkova et al. [16], and observing similar performances;
- We compress the model through different strategies, namely *Post Training Quantization* (PTQ) or *Quantization Aware Training* (QAT) using *High Granularity Quantization* (HGQ). We then port the results to FPGA, showing that the resource usage is well within the requirements of a L1T trigger system.

2 Data Samples

We use the datasets employed by [16], published on Zenodo [29, 30, 31, 32] and discussed in detail in [33].

The SM data sample represents a typical proton-proton collision dataset that has been pre-filtered by requiring the presence of an electron or a muon with a transverse momentum $p_T > 23 \text{ GeV}$ and a pseudorapidity $|\eta| < 3$ (electron) and $|\eta| < 2.1$ (muon). This is representative of a typical L1T selection algorithm of a multipurpose LHC experiment. In addition to

this, we consider the four benchmark new physics scenarios discussed in Cerri et al. [13]:

- A leptoquark (LQ) with a mass of 80 GeV, decaying to a b quark and a τ lepton: $LQ \rightarrow b\tau$ [30],
- A neutral scalar boson (A) with a mass of 50 GeV, decaying to two off-shell Z bosons, each forced to decay to two leptons: $A \rightarrow 4\ell$ [29],
- A scalar boson with a mass of 60 GeV, decaying to two tau leptons: $h^0 \rightarrow \tau\tau$ [31],
- A charged scalar boson with a mass of 60 GeV, decaying to a tau lepton and a neutrino: $h^\pm \rightarrow \tau\nu$ [32].

These four processes are used to evaluate the accuracy of the trained models.

In total, we use 3.5 million events from the background sample for training. The new physics benchmark samples are only used for evaluating the performance of the models, along with an additional 2 million background events not used in training.

Preprocessing We apply a simple standard-scaling operation in training and inference. This operation is usually replicated on FPGA through a bit-shift, as it involves just subtraction and division, and, while it implies the use of a limited amount of extra resources, it could theoretically be applied upstream in the board and be used for running multiple algorithms. Thus, it is not included in the resources estimate of the flow algorithm.

We note that the performance of the algorithm is very much dependent on the specific preprocessing operation being applied.

3 Normalizing Flow Models

Normalizing Flows (NFs) are a class of generative models that learn an explicit representation of an unknown data probability density function (pdf), $p(\mathbf{x})$. They achieve this by defining an invertible and differentiable mapping, f , between the complex data space and a simple base distribution (typically a standard Gaussian), $p(\mathbf{z})$:

$$\mathbf{x} = f(\mathbf{z}) \quad \text{and} \quad \mathbf{z} = f^{-1}(\mathbf{x}). \quad (1)$$

We refer to [20] for a comprehensive review of existing NF algorithms.

3.1 Continuous Normalizing Flows

A *Continuous Normalizing Flow* (CNF) defines the invertible mapping f not as a single function, but as the solution to a differential equation parameterized by a continuous variable $t \in [0, 1]$. This establishes a "probability path" that smoothly transforms the

base distribution p_0 into the target data distribution p_1 . The dynamics of this transformation are governed by a time-dependent vector field $\mathbf{v}_t$, which is parameterized by a neural network with parameters ϕ :

$$\frac{d\mathbf{z}(t)}{dt} = \mathbf{v}_t(\mathbf{z}(t), t|\phi), \quad (2)$$

with the initial condition $\mathbf{z}(0)$ being a sample from the base distribution, $\mathbf{z}(0) \sim p(\mathbf{z})$. A data-like sample $\mathbf{x}$ is generated by solving this *Ordinary Differential Equation* (ODE) from $t = 0$ to $t = 1$, yielding $\mathbf{x} = \mathbf{z}(1)$. Note that the dimensionality of the vector field $\mathbf{v}_t$ is the same as the input features $\mathbf{x}$.

3.2 Flow Matching

A powerful and stable method for training CNFs is *Flow Matching* [34, 35]. This approach recasts the training objective as a simple regression problem. The goal is to make the model's vector field $\mathbf{v}_t$ match a predefined target vector field $\mathbf{u}_t$. This target field $\mathbf{u}_t$ is constructed to transport samples from the base distribution towards samples from the data distribution along a specific "probability path" p_t .

Crucially, the path p_t and its corresponding vector field $\mathbf{u}_t$ can be constructed conditionally for each training sample $\mathbf{x}$. This simplifies the training into a regression of the model's output $\mathbf{v}_t$ onto the target $\mathbf{u}_t$ at random points along these paths. The loss function for the model parameters ϕ is then:

$$\mathcal{L}_{\text{FM}}(\phi) = \mathbb{E}_{t, \mathbf{x}, \mathbf{z}_t} \left[\|\mathbf{v}_t(\mathbf{z}_t, t|\phi) - \mathbf{u}_t(\mathbf{z}_t, t|\mathbf{x})\|^2 \right], \quad (3)$$

where $t \sim \text{Unif}[0, 1]$, $\mathbf{x} \sim p(\mathbf{x})$, and $\mathbf{z}_t$ is a point sampled from the conditional path distribution $p_t(\mathbf{z}|\mathbf{x})$. This is a straightforward regression loss, making the model relatively easy to train. In this work, we draw from recent developments reviewed in [35]. We refer the reader there for a detailed discussion on the construction of various probability paths and their associated vector fields.

3.3 Model architecture and training

The input to our model consists of the kinematic variables—transverse momentum (p_T), pseudorapidity (η), and azimuthal angle (ϕ)—for 18 reconstructed physics objects. These objects are ordered by type: 4 muons, 4 electrons, and 10 jets. The list is augmented with the missing transverse energy (MET), for which the magnitude and ϕ are used, while its η component is set to zero. This creates a fixed-size input tensor of shape (19, 3). For events with fewer than the maximum number of objects of a given type, the corresponding input slots are zero-padded, a standard practice in L1T algorithm design.

Before being processed by the network, this (19, 3) tensor is flattened into a single 57-dimensional vector. As detailed previously, a standard scaling transformation is applied to these input features. This pre-processing step is a simple affine transformation that can be readily implemented on an FPGA as a bit-shift operation; since its resource usage is minimal, it is omitted from the following discussion.

The core of our model is the neural network that parameterizes the time-dependent vector field $\mathbf{v}_t(\mathbf{z}_t, t|\phi)$ of the Continuous Normalizing Flow. After a basic hyperparameter scan, we settle for simple multilayer perceptron (MLP) architecture consisting of two hidden layers, each with 16 nodes, and using the Rectified Linear Unit (ReLU) as the activation function. The network takes a 58-dimensional flattened input vector, i.e. the 57 physics features plus an additional input for the current timestamp t , and outputs a 57-dimensional vector, which represents the vector field $\mathbf{v}_t$ required for the flow transformation. This simple architecture amounts to 1,913 trainable parameters.

The model was trained for 100 epochs using the Adam optimizer with a learning rate of 10^{-3} and a batch size of 1024. The network parameters were optimized by minimizing the Flow Matching loss described in section 3 (with the scheduler called `CondOTScheduler()` and the `AffineProbPath()` defined in [35] to build the loss function), using the PyTorch [36] and `flow-matching` [35] libraries.

3.4 VAE model for comparison

In order to compare to the fully-connected vae architecture of Ref. [16], we decide to retrain the same DNN VAE architecture. We refer the reader to section 3 of that work for details about the implementation; we train with the same conditions and the hyperparameter $\beta = 0.8$, plus the same preprocessing used for the flow architecture.

We could not exactly reproduce the results obtained in Ref. [16]. In addition to possible differences in the underlying training software, a possible cause of discrepancy could originate from undocumented pre-processing steps, with respect to the code published by the authors, see [37, 38]. In our experiments, we do observe a large performance dependence on the applied processing. For consistency reasons, we take our trained version of the VAE as the reference in our plots, but we still report the original results of [16] in the summary tables.

4 Anomaly Detection Scores

Once trained, Continuous Normalizing Flows map an input data point $\mathbf{x}$ to a latent variable $\mathbf{z}$ by solving the ODE that defines the model, see Eq. 2.

We denote as $\mathbf{v}_t(\mathbf{z}(t), t|\phi)$ the neural network with parameters ϕ that defines a time-dependent vector field. For this application, the integration runs from $t = 1$, where $\mathbf{z}(1) = \mathbf{x}$, to $t = 0$, where $\mathbf{z}(0)$ is the corresponding point in the latent Gaussian space.

Flow ODE The canonical approach to define an anomaly score for flows is the negative Gaussian log-likelihood of the latent point $\mathbf{z}(0)$. This score has a direct probabilistic interpretation, making it an elegant definition. However, calculating it requires integrating the ODE, a procedure too complex to implement efficiently on an FPGA. This would involve:

1. Starting from the data point $\mathbf{x}$ at timestep $t = 1$.
2. Executing a forward pass of the model to get the vector field $\mathbf{v}_t$.
3. Solving the current step of the ODE integrator using this vector field.
4. Repeating steps 2 and 3 until the integration reaches $t = 0$ to find the latent point $\mathbf{z}(0)$.
5. Evaluating the Gaussian log-likelihood of $\mathbf{z}(0)$ to get the final score.

This iterative process is clearly infeasible within the strict latency constraints of the L1T. In what follows, we still compute this score on a CPU with floating-point precision as a comparison, and we refer to it as *Flow ODE*. We show results obtained using the euler solver with just 2 timesteps from 1 to 0. From a preliminary investigation, when increasing the timesteps of integration performances are anyways similar to those showed in section 5 for the current choice of timesteps.

Flow $\mathbf{v}_t$ To create an FPGA-compatible alternative, we need an anomaly score that avoids this integration. We propose a new score based on the intuition that anomalous points, being further from the training distribution, require a larger "push" from the vector field to be mapped toward the latent prior. Therefore, the magnitude of the vector field itself, i.e. the output of the model, can serve as a proxy for anomaly. We define our score as the squared norm of the vector field evaluated at the initial point $\mathbf{x}$:

$$\mathcal{AS}(\mathbf{x}) = \|\mathbf{v}_t(\mathbf{x}, t|\phi)\|^2 = \sum_i (v_t(\mathbf{x}, t|\phi)_i)^2. \quad (4)$$

This strategy has the key advantage of requiring only a single forward pass of the model, making it suitable for fast inference. We refer to this approach as *Flow $\mathbf{v}_t$* .

The value of this score depends on the timestep t at which the model is evaluated. For this work, we choose to evaluate the field at the beginning of the trajectory, $t = 1$, as this is where the model first encounters the input data. We note that other choices are possible, and recent developments in single-step or straight-path flow models may offer other optimal evaluation points.

VAE Score In order to present a comparison with a similar architecture as the one used in [16], we use the Kullback-Leibler score D_{KL} used in the loss function as the anomaly score for this model:

$$D_{KL} = \frac{1}{2} \sum_{j=1}^2 (\sigma_j^2 + \mu_j^2 - \log(\sigma_j^2) - 1) \quad (5)$$

and we refer to it as *VAE* or D_{KL} in the following.

Further considerations and outlook Finally, we would like to note that a vector-field-based approach also opens avenues for other potential scores, such as the *divergence* of the field (at the cost of multiple model evaluations), and provides a conceptual link to other ODE-based generative models like *Diffusion Models* [39]. In this paper, however, we focus on the first two approaches described, leaving the investigation of these alternatives to future work.

5 Results at Floating Point precision

The model performance is assessed using the four new physics benchmark models. The anomaly detection scores considered are the *Flow $\mathbf{v}_t$* and *Flow ODE* (as offline comparison) for the flow model, and the D_{KL} for the VAE model.

The receiver operating characteristic (ROC) curves in Figure 1 show the true positive rate (TPR) as a function of the false positive rate (FPR), computed by changing the lower threshold applied on the different anomaly scores. We further quantify the AD performance quoting the area under the ROC curve (AUC) and the TPR corresponding to a FPR working point of 10^{-5} (see Table I), which on this dataset corresponds to the reduction of the background rate to approximately 1000 events per month. We show as a dashed red line the working point of 10^{-5} FPR; and as a dotted grey line the baseline performance of a random classifier.

Table 1 summarizes the results numerically for both ROC AUC score and TPR, reporting a comparison with the D_{KL} anomaly score from [16] as well.

Looking at the results, we see that the flow is capable of detecting anomalies when using either the $\mathbf{v}_t$ or the *ODE* anomaly scores. The *ODE* score has

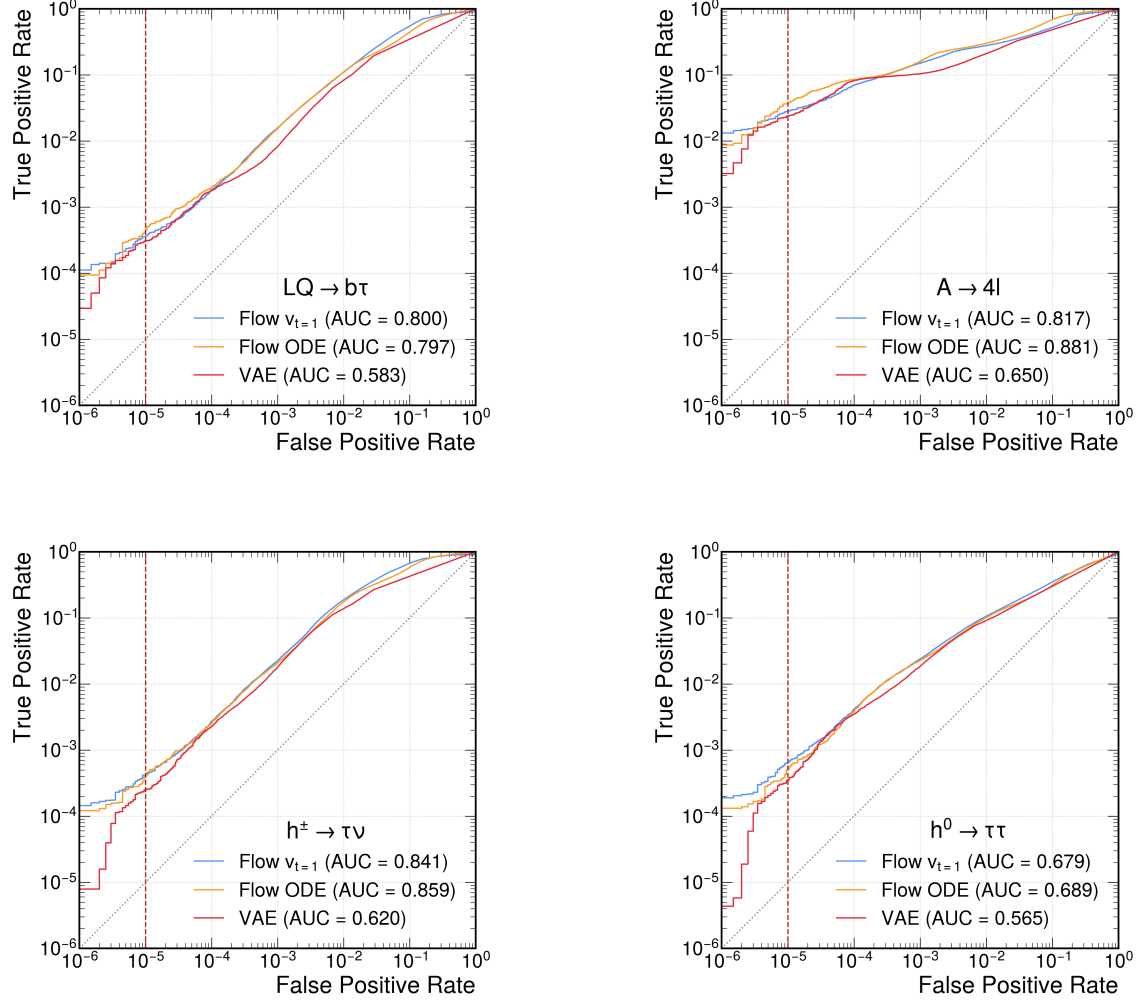


Figure 1: ROC curves for the 4 anomalous samples. For each sample we show the results for the flow, for both the Flow $\mathbf{v}_t$ and Flow ODE anomaly scores, and for the VAE. We show as a dashed red line the threshold of 10^{-5} FPR at which we evaluate the fraction of TPR for each model; and as a dotted grey line the baseline performance of a random classifier. The flow is capable of detecting the anomalies, with both ROC AUC and TPR better than a VAE trained under the same condition.

Table 1: Physics performance comparison of different anomaly detection models and scores. Performance is measured by TPR at a fixed FPR of 10^{-5} and the Area Under the ROC Curve (AUC).

Model	AD score	TPR @ FPR 10^{-5} [%]				AUC [%]			
		$LQ \rightarrow b\tau$	$A \rightarrow 4\ell$	$h^\pm \rightarrow \tau\nu$	$h^0 \rightarrow \tau\tau$	$LQ \rightarrow b\tau$	$A \rightarrow 4\ell$	$h^\pm \rightarrow \tau\nu$	$h^0 \rightarrow \tau\tau$
VAE from [16]	D_{KL}	0.07	5.27	0.08	0.11	92	94	94	81
Flow	$\mathbf{v}_t$	0.04	2.8	0.04	0.06	80	82	84	68
	ODE	0.04	3.8	0.04	0.05	80	88	86	69
VAE (Ours)	D_{KL}	0.02	2.4	0.02	0.03	58	65	62	57

marginally better ROC AUC scores, but similar performances when comparing the TPR values.

Interestingly, the VAE model achieves lower performance than the flow on all processes. As already noticed, the VAE performance does not match those shown in [16]. This could be due to differences in preprocessing with respect to the published dataset. This does not make our comparison less meaningful, since we used the same preprocessing for both our VAE implementation and the Flow model.

We attribute the differences between the model results in this work and the ones of [16] to:

1. A difference in the data partitions used for training;
2. A difference in the exact preprocessing operations used on the data (in which case it would be interesting to assess the performance of the flow under the same preprocessing);
3. Any remaining difference in the hyperparameters of the NN/training.

6 Model Compression and HLS C Simulation results

As mentioned in section 1, we perform two different compression strategies for porting the model to FPGA.

PTQ We perform some tests and find that a simple, brute force casting of the precision to 18 bits *post-training* is enough for synthesizing the algorithm while having a good tradeoff between performance and resource consumption. The actual weights of the network can be casted down to 12 bits, leaving the 18 bits casting for biases and other operations.

However, we observe that significantly less compression is achievable by default on the einsum operation, responsible for computing the norm squared of the model's output. We find that at least 23 bits are required for casting this operation and retain a good performance, meaning that the majority of resources spent and of the latency will be due to this operation alone.

Performance estimates from the HLS C Simulation of the PTQ model are reported in Table 2. For the same quantization, we report in Table 3 the resource usage, latency, and initiation interval for the PTQ model deployed on a Xilinx Virtex UltraScale+ FPGA. Resources are based on the Vivado estimates from Vivado HLS.

We observe a good retention of performance, with comparable results to the un-casted model on the TPR and minimal drops of performance on the ROC AUC. The resources being used are a fraction of the

available ones on the board, around 7%, making it possible to host this algorithm on FPGA along with other trigger algorithms, as is usually the case for these applications. The latency of 230 ns and the initiation interval of 5 ns make the algorithm fully compatible with the constraints from the trigger rates of a large LHC experiment and the LHC operational parameters of Run 3.

QAT The model trained with Quantization Aware Training (QAT) is trained using the High Granularity Quantization (HGQ) method [40].

We initialize the model with 6-fractional bits for weights and activations. We then train the model for 2000 epochs with a cosine decay restarting learning schedule to map out the Pareto front between performance and bit-width with increasing β from $5e-7$ to $5e-6$. Here, β is a hyperparameter that controls the trade-off between model performance and bit-width reduction during training, see [40] for details. We show one model obtained with this method in Table 2 and Table 3. Since HGQ quantizes the weights at a per-weight level and includes zero-bit (pruning), the final model is simultaneously unstructured pruned and quantized. The distribution of bit-widths for the weights of the model is shown in Figure 2.

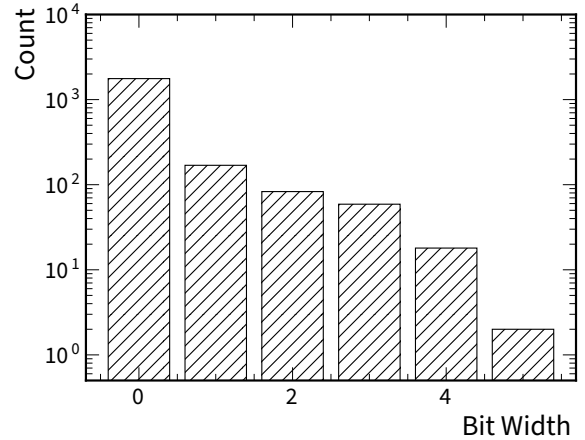


Figure 2: Distribution of bit-widths for the weights of the HGQ-trained model. 84.2% of the weights are quantized to 0 bits (pruned), while the remaining weights are distributed between 1 and 5 bits.

We use the da4ml [27] library to convert the model to RTL design and optimize the constant-matrix-vector multiplication operations (i.e., the dense layers of the MLP) using distributed arithmetic for lower resource usage and latency. Specifically, we convert directly to Verilog, and synthesize with Vivado 2025.1 with a target clock period of 5 ns.

We list in Table 2 the physics performance of this approach, while in Table 3 we report the usage of resources. We again observe a good retention of performance and a marked improvement in resource

usage over the PTQ model, with a latency of just 35 ns. All results reported are from RTL behavioral simulation of the RTL designs via Verilator [41], and all resource usages are from Vivado post place and route reports.

7 Conclusions

We have presented the first application of a Continuous Normalizing Flow for unsupervised anomaly detection within the demanding real-time environment of an LHC Level-1 Trigger system. The central innovation of this work is an FPGA-friendly anomaly score, $\mathcal{AS} = \|\mathbf{v}_t\|^2$, derived directly from the model’s vector field output, which circumvents the need for computationally expensive ODE integration. We demonstrated that this approach successfully identifies a variety of new physics signatures, achieving performance comparable to a VAE trained under identical conditions. Furthermore, we have shown used advanced quantization techniques to prove that the synthesized algorithm meets the stringent L1T constraints, with a latency ranging from 230 to 35 ns and minimal resource utilization.

This work opens several avenues for future investigation:

- Enhancing the discriminating power of the vector field by incorporating physics-motivated or agnostic constraints into the Flow Matching loss function (Eq. 3.2).
- Exploring alternative anomaly scores derived from the vector field, such as its divergence, and extending the methodology to other ODE-based generative models like Diffusion Models.
- Performing a comprehensive scan over model architectures and flow probability paths, to further optimize performance and resource efficiency.
- Investigating model pruning and knowledge distillation to potentially simplify the model for inference while preserving its performance.

We see Normalizing Flows as not just a useful tool for offline analysis, but also a practical one for deployment in real-time discovery. This study paves the way for integrating this new class of powerful, model-agnostic algorithms into the next generation of trigger systems, expanding the potential for model-agnostic searches at 40 MHz.

Acknowledgments

Francesco Vaselli acknowledges the support and guidance received to kick-off this work in the context of the first CERN Next Generation Triggers hackathon, April 7-11, 2025, CERN, Geneva.

The work done by Dimitrios Danopoulos, Roope Oskari Niemi and Felice Pantaleo is supported by the Eric & Wendy Schmidt Fund for Strategic Innovation through the CERN Next Generation Triggers project (grant agreement SIF- 2023-004).

Chang Sun is partially supported by the U.S. Department of Energy (DOE), Office of Science, Office of High Energy Physics grant DE-SC0011925, NSF ACCESS Grant PHY240298.

This work was supported by the Open Access Publishing Fund of the Scuola Normale Superiore.

X1: Code availability

The code used for this work is made publicly available under MIT License at the following URL: <https://github.com/francesco-vaselli/fAD>.

The datasets used are hosted on Zenodo as explained in section 2.

References

- [1] Lyndon Evans and Philip Bryant. “LHC Machine”. In: *Journal of Instrumentation* 3.08 (Aug. 2008), S08001. DOI: 10.1088/1748-0221/3/08/S08001. URL: <https://dx.doi.org/10.1088/1748-0221/3/08/S08001>.
- [2] The ATLAS Collaboration. “The ATLAS Experiment at the CERN Large Hadron Collider”. In: *Journal of Instrumentation* 3.08 (Aug. 2008), S08003. DOI: 10.1088/1748-0221/3/08/S08003. URL: <https://dx.doi.org/10.1088/1748-0221/3/08/S08003>.
- [3] The CMS Collaboration. “The CMS experiment at the CERN LHC”. In: *Journal of Instrumentation* 3.08 (Aug. 2008), S08004. DOI: 10.1088/1748-0221/3/08/S08004. URL: <https://dx.doi.org/10.1088/1748-0221/3/08/S08004>.
- [4] The CMS Collaboration. “Performance of the CMS Level-1 trigger in proton-proton collisions at $\sqrt{s} = 13$ TeV”. In: *Journal of Instrumentation* 15.10 (Oct. 2020), P10017–P10017. ISSN: 1748-0221. DOI: 10.1088/1748-0221/15/10/p10017. URL: <http://dx.doi.org/10.1088/1748-0221/15/10/P10017>.
- [5] *The Phase-2 Upgrade of the CMS Level-1 Trigger*. Tech. rep. Final version. Geneva: CERN, 2020. URL: <https://cds.cern.ch/record/2714892>.

Table 2: Physics performance comparison for the PTQ and HGQ model. Performance is measured by the TPR at a fixed FPR of 10^{-5} and the Area Under the ROC Curve (AUC). For the AUC we also report the ratio with the AUC score of the offline Flow $\mathbf{v}_t$ model.

Model	TPR @ FPR 10^{-5} [%]				AUC [%]			
	$LQ \rightarrow b\tau$	$A \rightarrow 4\ell$	$h^\pm \rightarrow \tau\nu$	$h^0 \rightarrow \tau\tau$	$LQ \rightarrow b\tau$	$A \rightarrow 4\ell$	$h^\pm \rightarrow \tau\nu$	$h^0 \rightarrow \tau\tau$
Flow PTQ	0.03	3.6	0.04	0.06	75 (0.94)	81 (0.92)	81 (0.94)	65 (0.94)
Flow HGQ	0.04	3.4	0.05	0.06	77 (0.96)	86 (0.97)	82 (0.95)	66 (0.96)

Table 3: Post-routing resource usage (as total estimates and as percentages over the whole board resources), latency, and initiation interval for the PTQ and HGQ model deployed on a Xilinx Virtex UltraScale+ xcu250-figd2104-2L-e FPGA. The designs are implemented Vivado 2025.1, with a target clock period of 5 ns out-of-context.

Model	DSP	LUT	FF	BRAM	Latency [ns]	II [clk]
Flow PTQ	916 (7.45%)	40,835 (2.36%)	11,397 (0.33%)	0	230	1
Flow HGQ	28 (<0.01%)	5,978 (0.34%)	1,683 (0.05%)	0	35	1

- [6] The ATLAS collaboration. “Operation of the ATLAS trigger system in Run 2”. In: *Journal of Instrumentation* 15.10 (Oct. 2020), P10004–P10004. ISSN: 1748-0221. DOI: 10.1088/1748-0221/15/10/p10004. URL: <http://dx.doi.org/10.1088/1748-0221/15/10/P10004>.
- [7] *Technical Design Report for the Phase-II Upgrade of the ATLAS TDAQ System*. Tech. rep. Geneva: CERN, 2017. DOI: 10.17181/CERN.2LBB.4IAL. URL: <https://cds.cern.ch/record/2285584>.
- [8] The ATLAS Collaboration. “Observation of a new particle in the search for the Standard Model Higgs boson with the ATLAS detector at the LHC”. In: *Physics Letters B* 716.1 (2012), pp. 1–29. ISSN: 0370-2693. DOI: <https://doi.org/10.1016/j.physletb.2012.08.020>. URL: <https://www.sciencedirect.com/science/article/pii/S037026931200857X>.
- [9] The CMS Collaboration. “Observation of a new boson at a mass of 125 GeV with the CMS experiment at the LHC”. In: *Physics Letters B* 716.1 (2012), pp. 30–61. ISSN: 0370-2693. DOI: <https://doi.org/10.1016/j.physletb.2012.08.021>. URL: <https://www.sciencedirect.com/science/article/pii/S0370269312008581>.
- [10] Thea Aarrestad et al. “The Dark Machines Anomaly Score Challenge: Benchmark Data and Model Independent Event Classification for the Large Hadron Collider”. In: *SciPost Physics* 12.1 (Jan. 2022). ISSN: 2542-4653. DOI: 10.21468/scipostphys.12.1.043. URL: <http://dx.doi.org/10.21468/SciPostPhys.12.1.043>.
- [11] Gregor Kasieczka et al. “The LHC Olympics 2020 a community challenge for anomaly detection in high energy physics”. In: *Reports on Progress in Physics* 84.12 (Dec. 2021), p. 124201. ISSN: 1361-6633. DOI: 10.1088/1361-6633/ac36b9. URL: <http://dx.doi.org/10.1088/1361-6633/ac36b9>.
- [12] Diederik P Kingma and Max Welling. *Auto-Encoding Variational Bayes*. 2022. arXiv: 1312.6114 [stat.ML]. URL: <https://arxiv.org/abs/1312.6114>.
- [13] Olmo Cerri et al. “Variational autoencoders for new physics mining at the Large Hadron Collider”. In: *Journal of High Energy Physics* 2019.5 (May 2019). ISSN: 1029-8479. DOI: 10.1007/jhep05(2019)036. URL: [http://dx.doi.org/10.1007/JHEP05\(2019\)036](http://dx.doi.org/10.1007/JHEP05(2019)036).
- [14] Oliver Knapp et al. *Adversarially Learned Anomaly Detection on CMS Open Data: re-discovering the top quark*. 2020. arXiv: 2005.01598 [hep-ex]. URL: <https://arxiv.org/abs/2005.01598>.
- [15] Francesco Poppi. “Is the bell ringing?” In: 46/2010 (2010), p. 14. URL: <https://cds.cern.ch/record/1306501>.
- [16] Ekaterina Govorkova et al. “Autoencoders on field-programmable gate arrays for real-time, unsupervised new physics detection at 40 MHz at the Large Hadron Collider”. In: *Nature Machine Intelligence* 4.2 (Feb. 2022), pp. 154–161. ISSN: 2522-5839. DOI: 10.1038/s42256-022-00441-3. URL: <http://dx.doi.org/10.1038/s42256-022-00441-3>.
- [17] Abhijith Gandrakota. *Real-time Anomaly Detection at the L1 Trigger of CMS Experiment*. 2024. arXiv: 2411.19506 [hep-ex]. URL: <https://arxiv.org/abs/2411.19506>.
- [18] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. *Distilling the Knowledge in a Neural Network*. 2015. arXiv: 1503.02531 [stat.ML]. URL: <https://arxiv.org/abs/1503.02531>.

- [19] Max Michael Cohen et al. “GELATO: a Generic Event-Level Anomalous Trigger Option for ATLAS - Slides”. In: (2025). URL: <https://cds.cern.ch/record/2938881>.
- [20] George Papamakarios et al. *Normalizing Flows for Probabilistic Modeling and Inference*. 2021. arXiv: 1912.02762 [stat.ML]. URL: <https://arxiv.org/abs/1912.02762>.
- [21] Claudius Krause et al. “Anomaly detection with flow-based fast calorimeter simulators”. In: *Physical Review D* 110.3 (Aug. 2024). ISSN: 2470-0029. DOI: 10.1103/physrevd.110.035036. URL: <http://dx.doi.org/10.1103/PhysRevD.110.035036>.
- [22] Pratik Jawahar et al. “Improving Variational Autoencoders for New Physics Detection at the LHC With Normalizing Flows”. In: *Frontiers in Big Data* 5 (Feb. 2022). ISSN: 2624-909X. DOI: 10.3389/fdata.2022.803685. URL: <http://dx.doi.org/10.3389/fdata.2022.803685>.
- [23] FastML Team. *fastmachinelearning/hls4ml*. Version v0.8.1. 2023. DOI: 10.5281/zenodo.1201549. URL: <https://github.com/fastmachinelearning/hls4ml>.
- [24] Javier Duarte et al. “Fast inference of deep neural networks in FPGAs for particle physics”. In: *JINST* 13.07 (2018), P07027. DOI: 10.1088/1748-0221/13/07/P07027. arXiv: 1804.06913 [physics.ins-det].
- [25] Thea Aarrestad et al. “Fast convolutional neural networks on FPGAs with hls4ml”. In: *Mach. Learn. Sci. Tech.* 2.4 (2021), p. 045015. DOI: 10.1088/2632-2153/ac0ea1. arXiv: 2101.05108 [cs.LG].
- [26] Jennifer Ngadiuba et al. “Compressing deep neural networks on FPGAs to binary and ternary precision with HLS4ML”. In: *Mach. Learn. Sci. Tech.* 2 (2021), p. 015001. DOI: 10.1088/2632-2153/aba042. arXiv: 2003.06308 [cs.LG].
- [27] Chang Sun et al. *da4ml: Distributed Arithmetic for Real-time Neural Networks on FPGAs*. 2025. arXiv: 2507.04535 [cs.AR]. URL: <https://arxiv.org/abs/2507.04535>.
- [28] Claudionor N. Coelho et al. “Automatic heterogeneous quantization of deep neural networks for low-latency inference on the edge for particle detectors”. In: *Nature Machine Intelligence* 3.8 (June 2021), pp. 675–686. ISSN: 2522-5839. DOI: 10.1038/s42256-021-00356-5. URL: <http://dx.doi.org/10.1038/s42256-021-00356-5>.
- [29] Thea Aarrestad et al. *Unsupervised New Physics detection at 40 MHz: A $\rightarrow 4$ leptons Signal Benchmark Dataset*. Zenodo, June 2021. DOI: 10.5281/zenodo.5046446. URL: <https://doi.org/10.5281/zenodo.5046446>.
- [30] Thea Aarrestad et al. *Unsupervised New Physics detection at 40 MHz: LQ $\rightarrow b$ tau Signal Benchmark Dataset*. Zenodo, July 2021. DOI: 10.5281/zenodo.5055454. URL: <https://doi.org/10.5281/zenodo.5055454>.
- [31] Thea Aarrestad et al. *Unsupervised New Physics detection at 40 MHz: $h^0 \rightarrow \tau \tau$ Signal Benchmark Dataset*. Zenodo, July 2021. DOI: 10.5281/zenodo.5061633. URL: <https://doi.org/10.5281/zenodo.5061633>.
- [32] Thea Aarrestad et al. *Unsupervised New Physics detection at 40 MHz: $h^{\pm} \rightarrow \tau \nu$ Signal Benchmark Dataset*. Zenodo, July 2021. DOI: 10.5281/zenodo.5061688. URL: <https://doi.org/10.5281/zenodo.5061688>.
- [33] Ekaterina Govorkova et al. *LHC physics dataset for unsupervised New Physics detection at 40 MHz*. 2021. arXiv: 2107.02157 [physics.data-an]. URL: <https://arxiv.org/abs/2107.02157>.
- [34] Yaron Lipman et al. *Flow Matching for Generative Modeling*. 2023. arXiv: 2210.02747 [cs.LG]. URL: <https://arxiv.org/abs/2210.02747>.
- [35] Yaron Lipman et al. *Flow Matching Guide and Code*. 2024. arXiv: 2412.06264 [cs.LG]. URL: <https://arxiv.org/abs/2412.06264>.
- [36] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. arXiv: 1912.01703 [cs.LG]. URL: <https://arxiv.org/abs/1912.01703>.
- [37] Ekaterina Govorkova et al. *fastml AE_L1_paper*. 2021. URL: https://github.com/fastmachinelearning/hls4ml/tree/AE_L1_paper.
- [38] Ekaterina Govorkova, Ema Puljak, and Thea et. al. Aarrestad. *cl-orca VAE AD retraining*. 2025. URL: <https://github.com/katyagovorkova/cl-orca/tree/main/paper/vae>.
- [39] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denosing Diffusion Probabilistic Models*. 2020. arXiv: 2006.11239 [cs.LG]. URL: <https://arxiv.org/abs/2006.11239>.
- [40] Sun Chang et al. *Gradient-based Automatic Per-Weight Mixed Precision Quantization for Neural Networks On-Chip*. en. 2024. DOI: 10.7907/HQ8JD-RHG30. URL: <https://authors.library.caltech.edu/doi/10.7907/hq8jd-rhg30>.

- [41] Wilson Snyder et al. *Verilator*. If you use this software, please cite it using the metadata from this file. URL: <https://verilator.org>.